Set Comprehensions

{expression for item in iterable if condition} – unique, unordered, hash-based results.

Basics

```
words = ["Apple", "apple", "BANANA",
         "banana", "Cherry"]
print(sorted({w.lower() for w in words}))
print(sorted({len(w) for w in words}))
nums = [1, 2, 2, 3, 3, 3]
print({n for n in nums if n % 2})
```

OUTPUT

```
['apple', 'banana', 'cherry']
[5, 6]
{1, 3}
```

Properties

Property	Meaning
unique	duplicates collapse automatically
unordered	no indexing; iteration order not guaranteed
hashable items	no lists / dicts / sets inside
fast membership	`x in s` is O(1) on average
mutable	`add`, `discard`, `update` all work

PITFALL `{}` is an empty **dict**. The empty set is `set()`.

```
print(type({}).__name__)
print(type({x for x in ()}).__name__)
```

OUTPUT

```
dict
set
```

Set algebra on the results

```
a = {n for n in range(1, 8) if n % 2}
b = {n for n in range(1, 8) if n % 3 == 0}
print(sorted(a), sorted(b))
print(sorted(a | b), sorted(a & b), sorted(a - b))
```

OUTPUT

```
[1, 3, 5, 7] [3, 6]
[1, 3, 5, 6, 7] [3] [1, 5, 7]
```

Operator	Meaning
`a | b`	union – in a or b
`a & b`	intersection – in both
`a - b`	difference – in a, not in b
`a ^ b`	symmetric difference – in exactly one
`a <= b`	subset test

Common uses

```
emails = ["a@x.com", "b@y.org", "c@x.com"]
domains = {e.split("@")[1] for e in emails}
print(sorted(domains))
print(sorted({ch for ch in "mississippi"}))

allowed = {n * n for n in range(100)}   # build once
# fast lookups
print(49 in allowed, 50 in allowed)
```

OUTPUT

```
['x.com', 'y.org']
['i', 'm', 'p', 's']
True False
```

Fast filtering against another collection

```
a, b = [1, 2, 3, 4], [3, 4, 5]
# O(1) membership
lookup = set(b)
common = {x for x in a if x in lookup}
print(sorted(common))

pts = {(x % 2, y % 2)
       for x in range(4) for y in range(4)}
# tuples are hashable
print(sorted(pts))
```

OUTPUT

```
[3, 4]
[(0, 0), (0, 1), (1, 0), (1, 1)]
```

Elements must be hashable

```
try:
    {[1, 2] for _ in range(1)}
except TypeError as e:
    print(e)
```

OUTPUT

```
unhashable type: 'list'
```

TIP Need to dedupe **and keep order**? Use `list(dict.fromkeys(items))`.

```
print(list(dict.fromkeys([3, 1, 3, 2, 1])))
```

OUTPUT

```
[3, 1, 2]
```